CRUD MySQL con Angular 22 — Guía paso a paso

Nov 5, 2026 · @Txema

Introducción

Esta guía reconstruye la app del repositorio jsersan/crudelectronica con **Angular 22** en el frontend y su **backend PHP (PDO) + MySQL** (`backend-php/`), siguiendo el orden, los nombres de ficheros y las pantallas del PDF *CRUD mySQL Angular*.

La app gestiona la tabla `productos` (alta, listado paginado, búsqueda por nombre, edición en diálogo, detalle y borrado) con Angular Material y Bootstrap.

Cuatro avisos antes de empezar:

1. **Backend.** El PDF construye un backend Node/Express en una carpeta `backEnd`; el repositorio no lo tiene. Lo sustituyó por una API REST en PHP que corre en Apache (MAMP en local, Arsys en producción), sin Node en el servidor. Esta guía usa ese backend PHP, con el código tal como está en el repo (versión de noviembre de 2026).
2. **Versión.** Angular 22 se publicó el 3 de junio de 2026 y tiene soporte activo hasta junio de 2027 y LTS hasta junio de 2028 (tabla oficial).
3. **Estado en signals.** En Angular 22 las apps nuevas son *zoneless* y los componentes usan `OnPush` por defecto (Ninja Squad, Angular 22.0). Por eso los datos que llegan del servidor se guardan en **signals**; la lógica es la del PDF.
4. **Lo que no cubre.** La traducción ES/EU, el interceptor de reintentos y el overlay de carga del repo quedan fuera para mantener la guía fiel al PDF.

Qué cambia respecto al PDF (resumen): backend PHP en lugar de Node, componentes *standalone* sin `app.module.ts`, `app.config.ts` sin `HttpClientModule`, servicios con `@Service()` e `inject()`, estado en signals, `@if` en lugar de `*ngIf` y nuevas directivas de botón de Material. La tabla completa está al final.

Requisitos previos

Necesitas Node.js 22.22.3+ y el CLI 22 para el frontend, y MAMP (Apache + PHP + MySQL) para el backend. Los rangos de Node y TypeScript son los de la tabla de compatibilidad de Angular.

Herramienta	Versión	Comprobación
Node.js	^22.22.3 || ^24.15.0 || ^26.0.0	`node -v`
Angular CLI	22.x	`ng version`
TypeScript	>=6.0.0 <6.1.0	lo instala el CLI
PHP	7.1 o superior, con `pdo_mysql` (lo trae MAMP)	`php -m | grep pdo_mysql`
MySQL / MariaDB	MAMP: Apache 8888, MySQL 8889	phpMyAdmin
Git	cualquiera reciente	`git --version`

El backend usa `?string`, tipos de retorno `void` y el operador `??`, por eso pide PHP 7.1 como mínimo.

Instalar Node con nvm y el CLI 22:

```
nvm install 22        # última 22.x; necesitas 22.22.3 o superior
nvm use 22
node -v
npm install -g @angular/cli@22
ng version
```

macOS Catalina (iMac 2012). Node 24 no funciona: sus binarios oficiales exigen macOS 13.5 (Node.js, migración v22 a v24) y fallan con `dyld: Symbol not found`. Node 22.21.0 sí arranca en tu equipo, pero es anterior a la 22.22.3 que pide Angular 22. Falta comprobar si la última 22.x también arranca y si `ng serve` compila. Si no, la opción es Angular 20 con Node 20 (versión anterior de esta guía, en el historial del documento), un entorno en la nube u otro equipo. El backend PHP no depende de Node y funciona igual.

Estructura del proyecto y carpeta de trabajo

Como en el repositorio, el proyecto Angular ocupa la raíz y el backend va dentro, en `backend-php/`. El proyecto vive dentro de `htdocs` de MAMP para que Apache sirva el backend sin copiar nada.

```
crudelectronica/                ← proyecto Angular 22 (raíz)
├── backend-php/                ← API REST en PHP (del repo)
|   ├── index.php               ← router REST + CORS
|   ├── productos.php           ← consultas PDO a la tabla productos
|   ├── db.php                  ← conexión PDO
```

```
|   ├── config.php               ← configuración por defecto (sin secretos)
|   ├── config.local.php         ← credenciales reales (NO va a Git)
|   ├── helpers.php              ← respuestas JSON
|   ├── htaccess                 ← renombrar a .htaccess
|   └── sql/crud.sql             ← esquema + datos de ejemplo
├── src/
|   ├── index.html, main.ts, styles.scss
|   ├── htaccess                 ← para producción (Paso 8)
|   ├── environments/
|   |   ├── environment.ts               ← producción
|   |   └── environment.development.ts   ← desarrollo (ng serve)
|   └── app/
|       ├── app.config.ts, app.routes.ts, app.ts, app.html
|       ├── models/producto.model.ts
|       ├── services/producto.service.ts
|       └── components/
|           ├── productos-list/
|           ├── producto-form/
|           └── producto-detalle/
├── angular.json
└── package.json
```

Crear la carpeta de trabajo

No merece la pena actualizar el repo de Angular 15 a 22 con `ng update`: habría que pasar versión a versión (15→16→…→22). Es más limpio crear un proyecto Angular 22 nuevo y traer el backend PHP del repo.

```
cd /Applications/MAMP/htdocs/angular22

# 1) Repo original, solo como referencia y para copiar el backend
git clone https://github.com/jsersan/crudelectronica.git crudelectronica-15

# 2) Proyecto Angular 22 nuevo (mismo nombre interno que el repo)
ng new crud-angular-mysql --directory crudelectronica --style=scss --
ssr=false --skip-tests

# 3) Copiar el backend PHP dentro del proyecto nuevo
cp -R crudelectronica-15/backend-php crudelectronica/
cd crudelectronica

# 4) Borrar los ficheros ._* y .DS_Store que se colaron en el repo
find backend-php -name '._*' -delete
find backend-php -name '.DS_Store' -delete
```

Si `angular22` no está dentro de `htdocs`, ajusta la ruta: lo que importa es que `backend-php` quede bajo `/Applications/MAMP/htdocs/`. En esta guía la URL local del backend será `http://localhost:8888/angular22/crudelectronica/backend-php`.

Si el CLI pregunta por herramientas de IA, responde *None* (o la que uses). No pregunta por zone.js: la app nace *zoneless*.

Paso 1: Configurar la base de datos MySQL

El script ya viene en el repo, en `backend-php/sql/crud.sql`: crea la base `crud_angular_mysql`, la tabla `productos` (la misma del PDF) y los cinco productos de ejemplo, todo en `utf8mb4`.

```
-- backend-php/sql/crud.sql (contenido del repositorio)

-- Fuerza la conexión de importación a utf8mb4. Sin esto, algunos clientes
-- (mysql CLI en latin1) doble-codifican tildes/ñ/euskera. Imprescindible.
SET NAMES utf8mb4;

CREATE DATABASE IF NOT EXISTS `crud_angular_mysql`
  CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
USE `crud_angular_mysql`;

CREATE TABLE IF NOT EXISTS `productos` (
  `id`          INT AUTO_INCREMENT PRIMARY KEY,
  `nombre`      VARCHAR(100) NOT NULL,
  `descripcion` TEXT,
  `precio`      DECIMAL(10,2) NOT NULL,
  `stock`       INT NOT NULL,
  `createdAt`   TIMESTAMP DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- --- Datos de ejemplo (opcional) ---
INSERT INTO `productos` (`nombre`, `descripcion`, `precio`, `stock`) VALUES
  ('Laptop Dell XPS 13',          'Laptop de alta gama con pantalla 4K',
1299.99, 15),
  ('iPhone 15 Pro',               'Último modelo con cámara avanzada',
999.99, 25),
  ('Monitor LG UltraWide',        'Monitor de 34 pulgadas curvo',
499.99, 10),
  ('Teclado Mecánico Logitech',   'Teclado con switches Cherry MX',
129.99, 30),
  ('Auriculares Sony WH-1000XM5', 'Cancelación de ruido premium',
349.99, 20);
```

1. Arranca MAMP (Apache 8888, MySQL 8889).
2. Importa el script desde la raíz del proyecto, con phpMyAdmin (*Importar*) o por terminal:

```
/Applications/MAMP/Library/bin/mysql -u root -p -h 127.0.0.1 -P 8889 < backend-php/sql/crud.sql
```

3. Comprueba con `SELECT * FROM productos;` que aparecen 5 filas, como en la pág. 29 del PDF.

Si ya tenías la base creada por el PDF, no hace falta repetirlo: la tabla es idéntica. Si importas dos veces, los productos de ejemplo se duplican (el `INSERT` no comprueba si ya existen).

Paso 2: Configurar el backend PHP

El backend del repo hace lo mismo que el Node/Express del PDF (mismas rutas y respuestas JSON), pero en PHP con PDO. No hay que instalar nada con npm: Apache de MAMP ejecuta los `.php` directamente.

2.1 Cómo está organizado

Cada petición entra por `index.php`, que aplica CORS, averigua la ruta y llama a `ProductoRepo` (`productos.php`); este usa la conexión de `db.php` con los datos de `config.php`, y `helpers.php` devuelve la respuesta en JSON. Equivalencia con el PDF: `index.php` = `server.js` + `producto.routes.js` + `producto.controller.js`; `productos.php` = `producto.model.js`; `db.php` = `db.js`; `config.php` = `.env` + `db.config.js`.

Método	Ruta	Qué hace	Respuesta
`GET`	`/productos`	Lista todos (orden `id ASC`)	`200` + array
`GET`	`/productos?nombre=texto`	Busca por nombre (contiene)	`200` + array
`GET`	`/productos/{id}`	Obtiene uno	`200` + objeto / `404`
`POST`	`/productos`	Crea	`201` + objeto creado

Método	Ruta	Qué hace	Respuesta
`PUT` / `PATCH`	`/productos/{id}`	Actualiza	`200` + objeto / `404`
`DELETE`	`/productos/{id}`	Borra uno	`200` + mensaje
`DELETE`	`/productos`	Borra **todos**	`200` + mensaje
`OPTIONS`	cualquiera	*Preflight* CORS	`204`

Acepta también el prefijo `/api/productos` (compatibilidad con el backend Node) y funciona sin `mod_rewrite` (`index.php/productos` o `?path=productos`). Los errores llegan como `{ "error": "mensaje" }`.

2.2 Configuración: `backend-php/config.php`

Importante: el `config.php` que está ahora en el repo contiene el host, el usuario y la contraseña reales de la base de datos de Arsys. Sustitúyelo por esta versión, que solo tiene los valores de MAMP, y lleva las credenciales reales a `config.local.php` (ya está en el `.gitignore`).

```
<?php
// backend-php/config.php · Valores por defecto para desarrollo (MAMP). SIN
secretos.
$config = [
    'db' => [
        'host'    => '127.0.0.1',   // 127.0.0.1 fuerza TCP; 'localhost'
usaría el socket
        'port'    => '8889',        // MySQL de MAMP
        'name'    => 'crud_angular_mysql',
        'user'    => 'root',
        'pass'    => 'root',
        'charset' => 'utf8mb4',
    ],
    'cors_origins' => ['http://localhost:4200'],
];

// Si existe config.local.php (servidor de producción), sobrescribe estos
valores
$local = __DIR__ . '/config.local.php';
if (is_file($local)) {
    $override = require $local;
    if (is_array($override)) {
        $config = array_replace_recursive($config, $override);
```

```
    }
}

return $config;
```

Plantilla de `config.local.php`, que solo existe en el servidor y **nunca** se sube a Git:

```
<?php
// backend-php/config.local.php · Credenciales reales (fuera de Git)
return [
    'db' => [
        'host' => 'servidor-mysql-de-arsys',
        'port' => '3306',
        'name' => 'nombre_bd',
        'user' => 'usuario',
        'pass' => 'contraseña',
    ],
];
```

La clave `cors_origins` no la usa nadie: los orígenes permitidos están fijos en `index.php` (apartado 2.6).

2.3 Conexión: `backend-php/db.php`

```
<?php
//
================================================================================
//  DB.PHP  ·  Conexión PDO a MySQL (compatible con PHP 7.0+; sin tipos 7.4)
//
================================================================================

class DB
{
    private static $pdo = null;
    private static $cfg = array();

    public static function init($config)
    {
        self::$cfg = $config;
    }

    public static function pdo()
    {
        if (self::$pdo === null) {
            $c   = self::$cfg['db'];
            $dsn = "mysql:host={$c['host']};port={$c['port']};dbname=
```

```
{$c['name']};charset={$c['charset']}";
            self::$pdo = new PDO($dsn, $c['user'], $c['pass'], array(
                PDO::ATTR_ERRMODE            => PDO::ERRMODE_EXCEPTION,
                PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
                PDO::ATTR_EMULATE_PREPARES   => false,
            ));
        }
        return self::$pdo;
    }
}
```

Una sola conexión por petición (patrón *singleton*), con excepciones activadas y consultas preparadas reales (`ATTR_EMULATE_PREPARES => false`).

2.4 Respuestas JSON: `backend-php/helpers.php`

```
<?php
//
// ============================================================================
//  HELPERS  ·  Respuestas JSON y utilidades de petición
//
// ============================================================================

class Response
{
    public static function json($data, int $code = 200): void
    {
        http_response_code($code);
        header('Content-Type: application/json; charset=utf-8');
        echo json_encode($data, JSON_UNESCAPED_UNICODE);
        exit;
    }

    public static function error(int $code, string $message): void
    {
        self::json(['error' => $message], $code);
    }
}

/** Lee y decodifica el cuerpo JSON de la petición. */
function body_json(): array
{
    $raw = file_get_contents('php://input');
```

```
    if ($raw === '' || $raw === false) return [];
    $data = json_decode($raw, true);
    return is_array($data) ? $data : [];
}
```

`JSON_UNESCAPED_UNICODE` deja las tildes tal cual (`Último`) en lugar de `\u00daltimo` . `exit` corta la ejecución tras responder, por eso el `switch` de `index.php` no necesita `break` .

2.5 Modelo: `backend-php/productos.php`

```
<?php
//
================================================================================
//  PRODUCTOS  ·  Repositorio relacional de la tabla `productos`
//  Devuelve objetos con el mismo shape que espera el frontend Angular:
//  { id:number, nombre, descripcion, precio:number, stock:number, createdAt
}
//
================================================================================

class ProductoRepo
{
    /** Normaliza una fila de BBDD al shape que consume el frontend. */
    private static function hydrate(array $row): array
    {
        return [
            'id'          => (int) $row['id'],
            'nombre'      => (string) $row['nombre'],
            'descripcion' => (string) $row['descripcion'],
            'precio'      => (float) $row['precio'],
            'stock'       => (int) $row['stock'],
            'createdAt'   => $row['createdAt'] ?? null,
        ];
    }

    /** Todos los productos. Si $nombre no es null, filtra por nombre (LIKE).
*/
    public static function all(?string $nombre = null): array
    {
        if ($nombre !== null && $nombre !== '') {
            $stmt = DB::pdo()->prepare(
                "SELECT id, nombre, descripcion, precio, stock, createdAt
                   FROM productos
                  WHERE nombre LIKE ?
                  ORDER BY id ASC"
```

```
            );
            $stmt->execute(['%' . $nombre . '%']);
        } else {
            $stmt = DB::pdo()->query(
                "SELECT id, nombre, descripcion, precio, stock, createdAt
                   FROM productos
                  ORDER BY id ASC"
            );
        }
        return array_map([self::class, 'hydrate'], $stmt->fetchAll());
    }

    public static function get(int $id): ?array
    {
        $stmt = DB::pdo()->prepare(
            "SELECT id, nombre, descripcion, precio, stock, createdAt
               FROM productos WHERE id = ?"
        );
        $stmt->execute([$id]);
        $row = $stmt->fetch();
        return $row ? self::hydrate($row) : null;
    }

    public static function create(array $data): array
    {
        $stmt = DB::pdo()->prepare(
            "INSERT INTO productos (nombre, descripcion, precio, stock)
             VALUES (:nombre, :descripcion, :precio, :stock)"
        );
        $stmt->execute(self::sanitize($data));
        return self::get((int) DB::pdo()->lastInsertId());
    }

    public static function update(int $id, array $data): ?array
    {
        $params = self::sanitize($data);
        $params['id'] = $id;
        $stmt = DB::pdo()->prepare(
            "UPDATE productos
                SET nombre = :nombre, descripcion = :descripcion,
                    precio = :precio, stock = :stock
              WHERE id = :id"
        );
        $stmt->execute($params);
        return self::get($id);
    }
```

```
    public static function delete(int $id): bool
    {
        $stmt = DB::pdo()->prepare("DELETE FROM productos WHERE id = ?");
        $stmt->execute([$id]);
        return $stmt->rowCount() > 0;
    }

    /** Borra todos los productos (equivale a DELETE /api/productos del
backend Node). */
    public static function deleteAll(): int
    {
        $stmt = DB::pdo()->query("DELETE FROM productos");
        return $stmt->rowCount();
    }

    /** Normaliza tipos. Permisivo, como el backend original (el formulario
     *  Angular ya valida obligatorios y longitudes en el cliente). */
    private static function sanitize(array $data): array
    {
        return [
            'nombre'      => trim((string) ($data['nombre'] ?? '')),
            'descripcion' => trim((string) ($data['descripcion'] ?? '')),
            'precio'      => (float) ($data['precio'] ?? 0),
            'stock'       => (int) ($data['stock'] ?? 0),
        ];
    }
}
```

`hydrate()` convierte `precio` a número: así el frontend recibe `600` y no `"600.00"` (el problema que en Node se resolvía con `decimalNumbers`). Todas las consultas son preparadas, también la búsqueda con `LIKE`, que en el PDF era vulnerable a inyección SQL.

2.6 Router: `backend-php/index.php`

```
<?php
//
// ============================================================================
//  INDEX.PHP  ·  Front controller / router REST  (CRUD Electrónica)
//  - CORS lo primero (para que el preflight OPTIONS responda siempre).
//  - Parseo de ruta compatible con PHP 7/8 y tolerante a que NO haya
//    mod_rewrite (admite /productos, index.php/productos y ?path=productos).
//
// ============================================================================
```

```
declare(strict_types=1);

// ---------- CORS (antes de cargar nada más) ----------
$__allowed = [
    'http://localhost:4200', 'http://127.0.0.1:4200',
    'http://txemaserrano.com', 'https://txemaserrano.com',
];
$__origin  = $_SERVER['HTTP_ORIGIN'] ?? '';
header('Access-Control-Allow-Origin: ' . (in_array($__origin, $__allowed,
true) ? $__origin : '*'));
header('Access-Control-Allow-Methods: GET, POST, PUT, PATCH, DELETE,
OPTIONS');
header('Access-Control-Allow-Headers: Content-Type, Authorization');
header('Vary: Origin');
if (($_SERVER['REQUEST_METHOD'] ?? 'GET') === 'OPTIONS') {
    http_response_code(204);
    exit;
}

$config = require __DIR__ . '/config.php';
require __DIR__ . '/helpers.php';
require __DIR__ . '/db.php';
require __DIR__ . '/productos.php';

DB::init($config);

$method = $_SERVER['REQUEST_METHOD'] ?? 'GET';

// ---------- Resolver la ruta ----------
$uri    = parse_url($_SERVER['REQUEST_URI'] ?? '/', PHP_URL_PATH) ?? '/';
$script = $_SERVER['SCRIPT_NAME'] ?? '';        // p.ej. /.../backend-
php/index.php
$base   = rtrim(dirname($script), '/');         // p.ej. /.../backend-php
$path   = $uri;
if ($base !== '' && strpos($path, $base) === 0) {
    $path = substr($path, strlen($base));
}
$path = ltrim($path, '/');
if (strpos($path, 'index.php') === 0) {         // sin mod_rewrite:
index.php/...
    $path = ltrim(substr($path, strlen('index.php')), '/');
}
if (isset($_GET['path'])) {                     // alternativa explícita: ?
path=...
    $path = trim((string) $_GET['path'], '/');
```

```
}
$path     = trim($path, '/');
$segments = $path === '' ? [] : explode('/', $path);

// Acepta un prefijo opcional "api/" (compatibilidad con /api/productos).
if (($segments[0] ?? '') === 'api') {
    array_shift($segments);
}

try {
    $recurso = $segments[0] ?? '';

    if ($recurso !== 'productos') {
        Response::error(404, 'Ruta no encontrada');
    }

    $id = isset($segments[1]) ? (int) $segments[1] : null;

    switch ($method) {
        case 'GET':
            if ($id) {
                $doc = ProductoRepo::get($id);
                $doc ? Response::json($doc) : Response::error(404, 'Producto
no encontrado');
            }
            $nombre = isset($_GET['nombre']) ? (string) $_GET['nombre'] :
null;
            Response::json(ProductoRepo::all($nombre));

        case 'POST':
            Response::json(ProductoRepo::create(body_json()), 201);

        case 'PUT':
        case 'PATCH':
            if (!$id) Response::error(400, 'Falta el id');
            $upd = ProductoRepo::update($id, body_json());
            $upd ? Response::json($upd) : Response::error(404, 'Producto no
encontrado');

        case 'DELETE':
            if (!$id) {                                  // DELETE /productos
→ borra todos
                ProductoRepo::deleteAll();
                Response::json(['message' => 'Todos los productos fueron
eliminados correctamente!']);
            }
```

```
            ProductoRepo::delete($id);
            Response::json(['message' => 'Producto eliminado
correctamente!']);

        default:
            Response::error(405, 'Método no permitido');
    }
} catch (PDOException $e) {
    Response::error(500, 'Error de base de datos: ' . $e->getMessage());
} catch (Throwable $e) {
    Response::error(500, 'Error: ' . $e->getMessage());
}
```

Tres detalles del código actual que conviene conocer: si el origen no está en `$__allowed`, CORS responde `*` (acepta cualquiera); `DELETE /productos/{id}` responde `200` aunque el id no exista (no comprueba lo que devuelve `delete()`); y los errores `500` incluyen el mensaje de PDO, que puede revelar detalles internos. Para clase sirve tal cual; en producción conviene corregir los tres.

2.7 Reescritura de URLs: `backend-php/.htaccess`

El repo guarda el fichero sin punto (`htaccess`) para que no se oculte. Renómbralo:

```
mv backend-php/htaccess backend-php/.htaccess
```

Contenido (del repo):

```
# Reescribe todas las peticiones al front controller index.php
RewriteEngine On
RewriteCond %{REQUEST_FILENAME} !-f
RewriteCond %{REQUEST_FILENAME} !-d
RewriteRule ^ index.php [QSA,L]

# Asegura que la cabecera Authorization llega a PHP (algunos Apache la
filtran)
RewriteCond %{HTTP:Authorization} .
RewriteRule .* - [E=HTTP_AUTHORIZATION:%{HTTP:Authorization}]
```

Con él, `.../backend-php/productos/3` funciona; sin él, usa `.../backend-php/index.php/productos/3`.

2.8 Probar la API

Con MAMP arrancado:

```
API=http://localhost:8888/angular22/crudelectronica/backend-php

curl "$API/productos"
curl "$API/productos?nombre=sony"
curl "$API/productos/3"
curl -X POST "$API/productos" -H "Content-Type: application/json" \
     -d '{"nombre":"Webcam
HD","descripcion":"1080p","precio":49.99,"stock":12}'
curl -X PUT "$API/productos/6" -H "Content-Type: application/json" \
     -d '{"nombre":"Webcam
HD","descripcion":"1080p","precio":44.99,"stock":20}'
curl -X DELETE "$API/productos/6"
```

El primero debe devolver el array JSON con los 5 productos. Si responde `404 Ruta no encontrada`, prueba `"$API/index.php/productos"`: el `.htaccess` no se está aplicando. Si responde `500 Error de base de datos`, revisa host, puerto y contraseña en `config.php`.

Paso 3: Preparar el frontend en Angular 22

El proyecto ya está creado (apartado *Crear la carpeta de trabajo*); aquí se configura para que se parezca al PDF y hable con el backend PHP.

3.1 Qué ha generado el CLI 22

Elemento	PDF (Angular 16) / repo (Angular 15)	CLI 22
Módulo raíz	`app.module.ts`	No existe: `app.config.ts`
Rutas	`app-routing.module.ts`	`app.routes.ts`
Componente raíz	`app.component.ts` / `AppComponent`	`app.ts` / `App`
Detección de cambios	zone.js + `Default`	Sin zone.js + `OnPush` por defecto
Entornos	`environment.ts` (desarrollo) + `environment.prod.ts`	Se crean en 3.7: `environment.development.ts` + `environment.ts` (producción)
Ficheros estáticos	`src/assets/`	`public/`

Elemento	PDF (Angular 16) / repo (Angular 15)	CLI 22
Tests	Karma + Jasmine	Vitest (omitidos con `--skip-tests`)

3.2 Recuperar los nombres de fichero del PDF

Desde Angular 20, `ng generate component productos-list` crea `productos-list.ts` con la clase `ProductosList`, sin sufijo. Para obtener `productos-list.component.ts` y `ProductosListComponent` como en el PDF y en el repo, añade en `angular.json`, dentro de `projects → crud-angular-mysql → schematics` (sustituye el bloque que ya existe):

```
"schematics": {
  "@schematics/angular:component": {
    "style": "scss",
    "type": "component",
    "skipTests": true
  },
  "@schematics/angular:service": {
    "type": "service",
    "skipTests": true
  }
}
```

Alternativa sin tocar `angular.json`: añadir `--type=component` o `--type=service` en cada `ng generate` (apaño citado en esta incidencia del CLI). El componente raíz `app.ts` se queda sin sufijo.

3.3 Instalar Bootstrap y Angular Material

```
npm install bootstrap@5 @angular/material@22 @angular/cdk@22
```

- Material y CDK deben ir en la misma versión mayor que `@angular/core`; así no hace falta el `.npmrc` con `legacy-peer-deps` que llegó a tener el repo.
- No se instala `@angular/animations`: Material no lo necesita desde la v20 y la API de animaciones está obsoleta desde Angular 20.2.
- Uso `npm install` y no `ng add @angular/material`, porque `ng add` reescribe `styles.scss` con un tema Material 3; así conservamos el tema `indigo-pink` del PDF y del repo.

3.4 Registrar los estilos en `angular.json`

En `projects → crud-angular-mysql → architect → build → options → styles`:

```
"styles": [
  "node_modules/bootstrap/dist/css/bootstrap.min.css",
  "@angular/material/prebuilt-themes/indigo-pink.css",
  "src/styles.scss"
]
```

El repo importa los dos CSS con `@import` en `styles.scss`, que Sass ha marcado como obsoleto. He comprobado que `indigo-pink.css` (Material Design 2) se publica hasta Material 21.2; no he podido abrir el paquete de la v22. Si `ng serve` no lo encuentra, usa `@angular/material/prebuilt-themes/azure-blue.css` (Material Design 3): cambia el aspecto, no el funcionamiento.

3.5 Estilos globales: `src/styles.scss`

```
/* src/styles.scss */
html, body { height: 100%; }
body { margin: 0; font-family: Roboto, "Helvetica Neue", sans-serif; }

/* En los componentes MDC la clase es .mat-mdc-form-field (no .mat-form-
field) */
.mat-mdc-form-field {
  margin-bottom: 15px;
}
```

3.6 Fuente de iconos: `src/index.html`

En el PDF los iconos salían como texto ("ed", "de") hasta añadir la fuente Material Icons (pág. 35). Aquí la ponemos desde el principio, con `lang="es"` y un título descriptivo (dos mejoras pendientes que lista el README del repo):

```
<!doctype html>
<html lang="es">
<head>
  <meta charset="utf-8">
  <title>CRUD Electrónica</title>
  <base href="/">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <link rel="icon" type="image/x-icon" href="favicon.ico">
  <link rel="preconnect" href="https://fonts.gstatic.com">
  <link href="https://fonts.googleapis.com/css2?
family=Roboto:wght@300;400;500&display=swap" rel="stylesheet">
  <link href="https://fonts.googleapis.com/icon?family=Material+Icons"
rel="stylesheet">
</head>
<body>
  <app-root></app-root>
</body>
</html>
```

3.7 URL del backend PHP en `environments`

```
ng generate environments
```

Crea `src/environments/environment.ts` y `environment.development.ts`, y añade el `fileReplacements` en `angular.json`. **Ojo, los nombres están al revés que en el repo:** en Angular 22 `environment.ts` es el de producción y `ng serve` usa `environment.development.ts`. El servicio añade `/productos` a `apiUrl`.

```
// src/environments/environment.development.ts  (ng serve)
// Backend PHP en MAMP: Apache en el puerto 8888
export const environment = {
  production: false,
  apiUrl: 'http://localhost:8888/angular22/crudelectronica/backend-php'
};
```

```
// src/environments/environment.ts  (ng build, producción en Arsys)
// Apunta directamente a index.php: funciona aunque el servidor no aplique el
.htaccess
export const environment = {
  production: true,
  apiUrl: '/angular/crudelectronicasimplificado-backend/index.php'
};
```

En el repo, `environment.prod.ts` apunta a `/angular/crudmysqlelebiduna-backend/index.php`, que parece heredado del proyecto *elebiduna*. Pon la carpeta real donde subas el backend (el ejemplo usa `crudelectronicasimplificado-backend`); el Paso 8 explica qué más debe coincidir.

Paso 4: Modelo de datos y servicio

El modelo es idéntico al del PDF. El servicio tiene los mismos métodos, pero usa el decorador `@Service()` de Angular 22 con `inject()`, la URL sale de `environment` y la búsqueda usa `HttpParams` para codificar el texto.

4.1 Modelo: `src/app/models/producto.model.ts`

```
ng generate interface models/producto model
```

El segundo argumento (`model`) es el tipo y produce el nombre `producto.model.ts`.

```
// src/app/models/producto.model.ts
export interface Producto {
  id?: number;
  nombre: string;
  descripcion: string;
  precio: number;
  stock: number;
  createdAt?: Date;
}
```

4.2 Servicio: `src/app/services/producto.service.ts`

```
ng generate service services/producto
```

En Angular 22 este comando genera la clase con `@Service()`, el equivalente abreviado de `@Injectable({ providedIn: 'root' })`. Su única condición es que las dependencias se obtengan con `inject()`, no por el constructor (Ninja Squad, Angular 22.0).

```
// src/app/services/producto.service.ts

// Service: decorador de Angular 22 (servicio disponible en toda la app)
// inject: obtiene dependencias sin usar el constructor
import { inject, Service } from '@angular/core';
// HttpClient para las peticiones; HttpParams para los parámetros de consulta
import { HttpClient, HttpParams } from '@angular/common/http';
// Observable para manejar operaciones asíncronas
```

```
import { Observable } from 'rxjs';
// Modelo Producto
import { Producto } from '../models/producto.model';
// URL base según el entorno (desarrollo / producción)
import { environment } from '../../environments/environment';

/**
 * Servicio para todas las operaciones CRUD de productos.
 * @Service() = @Injectable({ providedIn: 'root' })
 */
@Service()
export class ProductoService {
  // HttpClient ya viene incluido en Angular 22: no hace falta
provideHttpClient()
  private readonly http = inject(HttpClient);

  // URL base de la API, p. ej.
http://localhost:8888/angular22/crudelectronica/backend-php/productos
  private readonly apiUrl = `${environment.apiUrl}/productos`;

  /** Obtiene todos los productos */
  getAll(): Observable<Producto[]> {
    return this.http.get<Producto[]>(this.apiUrl);
  }

  /** Obtiene un producto por su ID */
  get(id: number): Observable<Producto> {
    return this.http.get<Producto>(`${this.apiUrl}/${id}`);
  }

  /** Crea un nuevo producto */
  create(data: Producto): Observable<Producto> {
    return this.http.post<Producto>(this.apiUrl, data);
  }

  /** Actualiza un producto existente */
  update(id: number, data: Producto): Observable<Producto> {
    return this.http.put<Producto>(`${this.apiUrl}/${id}`, data);
  }

  /** Elimina un producto por su ID */
  delete(id: number): Observable<unknown> {
    return this.http.delete(`${this.apiUrl}/${id}`);
  }

  /** Busca productos cuyo nombre contenga el texto indicado */
```

```
  findByNombre(nombre: string): Observable<Producto[]> {
    const params = new HttpParams().set('nombre', nombre);
    return this.http.get<Producto[]>(this.apiUrl, { params });
  }
}
```

Si el CLI te genera `@Injectable({ providedIn: 'root' })` en lugar de `@Service()`, déjalo: funciona igual. Para clase, mantengo Observables porque son los del PDF; Angular 22 también ofrece `httpResource()` (ya estable), que encaja con signals pero cambia más el código.

HttpClient usa ahora la API Fetch por defecto en lugar de XMLHttpRequest; para este CRUD no cambia nada (solo afecta al progreso de subida de ficheros).

Paso 5: Componentes

Los tres componentes del PDF mantienen nombre, métodos y plantilla; cambia dónde se guarda el estado. Con *zoneless* + `OnPush`, Angular solo repinta un componente cuando cambia un signal que lee su plantilla, cuando cambia un `@Input` o tras un evento del propio componente. Una respuesta HTTP que asigna una propiedad normal no dispara nada, así que **todo lo que llega del servidor se guarda en un signal**.

```
ng generate component components/productos-list
ng generate component components/producto-form
ng generate component components/producto-detalle
```

Con la configuración del apartado 3.2, cada carpeta tendrá `*.component.ts`, `*.component.html` y `*.component.scss`.

5.1 Lista: `productos-list.component.ts`

La paginación ya no usa `MatTableDataSource`: los productos están en un signal y un `computed` calcula la página visible. Es la forma más segura de que tabla y paginador se actualicen sin zone.js.

```
// src/app/components/productos-list/productos-list.component.ts

// Angular
import { Component, computed, inject, OnInit, signal } from '@angular/core';
import { CurrencyPipe } from '@angular/common';
import { FormsModule } from '@angular/forms'; // para [(ngModel)] del
buscador
// Angular Material
import { MatPaginatorModule, PageEvent } from '@angular/material/paginator';
```

```
import { MatTableModule } from '@angular/material/table';
import { MatDialog } from '@angular/material/dialog';
import { MatSnackBar } from '@angular/material/snack-bar';
import { MatFormFieldModule } from '@angular/material/form-field';
import { MatInputModule } from '@angular/material/input';
import { MatButtonModule } from '@angular/material/button';
import { MatIconModule } from '@angular/material/icon';
// Modelo, servicio y formulario que se abre en el diálogo
import { Producto } from '../../models/producto.model';
import { ProductoService } from '../../services/producto.service';
import { ProductoFormComponent } from '../producto-form/producto-
form.component';

/**
 * Lista de productos con búsqueda, paginación y acciones.
 * Standalone + OnPush (por defecto en Angular 22).
 */
@Component({
  selector: 'app-productos-list',
  imports: [
    CurrencyPipe,
    FormsModule,
    MatTableModule,
    MatPaginatorModule,
    MatFormFieldModule,
    MatInputModule,
    MatButtonModule,
    MatIconModule
  ],
  templateUrl: './productos-list.component.html',
  styleUrl: './productos-list.component.scss'
})
export class ProductosListComponent implements OnInit {
  // Dependencias con inject() (sustituye a los parámetros del constructor)
  private readonly productoService = inject(ProductoService);
  private readonly dialog = inject(MatDialog);
  private readonly snackBar = inject(MatSnackBar);

  // Columnas que se mostrarán en la tabla
  readonly displayedColumns: string[] = ['id', 'nombre', 'descripcion',
'precio', 'stock', 'acciones'];

  // Estado en signals: al cambiar, Angular repinta la plantilla
  readonly productos = signal<Producto[]>([]);
  readonly pageIndex = signal(0);
  readonly pageSize = signal(5);
```

```
  // Página visible = trozo del array según página y tamaño
  readonly paginaActual = computed(() => {
    const inicio = this.pageIndex() * this.pageSize();
    return this.productos().slice(inicio, inicio + this.pageSize());
  });

  // Término de búsqueda (lo actualiza el propio input, no hace falta signal)
  nombreBusqueda = '';

  /** Al iniciar el componente cargamos los productos */
  ngOnInit(): void {
    this.obtenerProductos();
  }

  /** Obtiene todos los productos */
  obtenerProductos(): void {
    this.productoService.getAll().subscribe({
      next: data => this.cargar(data),
      error: e => console.error(e)
    });
  }

  /** Busca por nombre; si el campo está vacío, recarga todo */
  buscarPorNombre(): void {
    const texto = this.nombreBusqueda.trim();
    this.pageIndex.set(0); // volver a la página 1 tras filtrar

    if (texto) {
      this.productoService.findByNombre(texto).subscribe({
        next: data => this.cargar(data),
        error: e => console.error(e)
      });
    } else {
      this.obtenerProductos();
    }
  }

  /** Evento (page) del paginador: guarda página y tamaño */
  cambiarPagina(evento: PageEvent): void {
    this.pageIndex.set(evento.pageIndex);
    this.pageSize.set(evento.pageSize);
  }

  /** Abre el formulario en un diálogo (crear si no hay producto, editar si
lo hay) */
```

```
  abrirFormulario(producto?: Producto): void {
    const dialogRef = this.dialog.open(ProductoFormComponent, {
      width: '500px',
      data: producto || {}
    });

    // Si el diálogo se cerró con resultado (no se canceló), recargamos
    dialogRef.afterClosed().subscribe(result => {
      if (result) {
        this.obtenerProductos();
      }
    });
  }

  /** Elimina un producto tras confirmarlo */
  eliminarProducto(id: number): void {
    if (confirm('¿Está seguro de eliminar este producto?')) {
      this.productoService.delete(id).subscribe({
        next: () => {
          this.snackBar.open('Producto eliminado correctamente', 'Cerrar', {
duration: 3000 });
          this.obtenerProductos();
        },
        error: e => console.error(e)
      });
    }
  }

  /** Guarda los datos y corrige la página si se ha quedado vacía (p. ej.
tras borrar) */
  private cargar(data: Producto[]): void {
    this.productos.set(data);
    const ultimaPagina = Math.max(0, Math.ceil(data.length / this.pageSize())
- 1);
    if (this.pageIndex() > ultimaPagina) {
      this.pageIndex.set(ultimaPagina);
    }
  }
}
```

5.2 Lista: `productos-list.component.html`

```
<!-- src/app/components/productos-list/productos-list.component.html -->

<div class="container mt-4">
  <h2>Lista de Productos</h2>
```

```
  <!-- Fila con el buscador y el botón de nuevo producto -->
  <div class="row mb-3">
    <div class="col-md-6">
      <mat-form-field appearance="outline" class="w-100">
        <mat-label>Buscar por nombre</mat-label>
        <!-- [(ngModel)] requiere FormsModule; Enter lanza la búsqueda -->
        <input matInput [(ngModel)]="nombreBusqueda"
(keyup.enter)="buscarPorNombre()">
        <!-- Solo mat-icon-button: con mat-button a la vez falla (NG0300 en
el PDF, NG8023 al compilar en v22) -->
        <button mat-icon-button matSuffix (click)="buscarPorNombre()" aria-
label="Buscar">
          <mat-icon>search</mat-icon>
        </button>
      </mat-form-field>
    </div>

    <div class="col-md-6 text-end">
      <!-- matButton="elevated" es la forma nueva de mat-raised-button -->
      <button matButton="elevated" color="primary"
(click)="abrirFormulario()">
        <mat-icon>add</mat-icon> Nuevo Producto
      </button>
    </div>
  </div>

  <!-- Tabla con sombra: la fuente de datos es la página calculada -->
  <div class="mat-elevation-z8">
    <table mat-table [dataSource]="paginaActual()" class="w-100">

      <ng-container matColumnDef="id">
        <th mat-header-cell *matHeaderCellDef>ID</th>
        <td mat-cell *matCellDef="let producto">{{ producto.id }}</td>
      </ng-container>

      <ng-container matColumnDef="nombre">
        <th mat-header-cell *matHeaderCellDef>Nombre</th>
        <td mat-cell *matCellDef="let producto">{{ producto.nombre }}</td>
      </ng-container>

      <ng-container matColumnDef="descripcion">
        <th mat-header-cell *matHeaderCellDef>Descripción</th>
        <td mat-cell *matCellDef="let producto">{{ producto.descripcion }}
</td>
      </ng-container>
```

```
      <ng-container matColumnDef="precio">
        <th mat-header-cell *matHeaderCellDef>Precio</th>
        <td mat-cell *matCellDef="let producto">{{ producto.precio | currency
}}</td>
      </ng-container>

      <ng-container matColumnDef="stock">
        <th mat-header-cell *matHeaderCellDef>Stock</th>
        <td mat-cell *matCellDef="let producto">{{ producto.stock }}</td>
      </ng-container>

      <!-- Acciones: editar y eliminar -->
      <ng-container matColumnDef="acciones">
        <th mat-header-cell *matHeaderCellDef>Acciones</th>
        <td mat-cell *matCellDef="let producto">
          <button mat-icon-button color="primary"
(click)="abrirFormulario(producto)" aria-label="Editar">
            <mat-icon>edit</mat-icon>
          </button>
          <button mat-icon-button color="warn"
(click)="eliminarProducto(producto.id)" aria-label="Eliminar">
            <mat-icon>delete</mat-icon>
          </button>
        </td>
      </ng-container>

      <tr mat-header-row *matHeaderRowDef="displayedColumns"></tr>
      <tr mat-row *matRowDef="let row; columns: displayedColumns;"></tr>

      <!-- Fila que se muestra cuando no hay datos -->
      <tr class="mat-row" *matNoDataRow>
        <td class="mat-cell text-center" colspan="6">No se encontraron
productos</td>
      </tr>
    </table>

    <!-- Paginador controlado por los signals del componente -->
    <mat-paginator
      [length]="productos().length"
      [pageIndex]="pageIndex()"
      [pageSize]="pageSize()"
      [pageSizeOptions]="[5, 10, 25]"
      (page)="cambiarPagina($event)"
```

```
      showFirstLastButtons>
    </mat-paginator>
  </div>
</div>
```

`*matCellDef`, `*matHeaderRowDef` y `*matNoDataRow` siguen con asterisco: son directivas de Material, no las `*ngIf` / `*ngFor` obsoletas.

5.3 Formulario: `producto-form.component.ts`

Sigue siendo un formulario reactivo (el del PDF); Angular 22 ofrece también Signal Forms, ya estables, pero cambiaría todo el componente. `MatDialogRef` y `MAT_DIALOG_DATA` se inyectan como opcionales para que la ruta `/add` del PDF funcione fuera del diálogo.

```
// src/app/components/producto-form/producto-form.component.ts

import { Component, inject } from '@angular/core';
import { FormBuilder, ReactiveFormsModule, Validators } from
'@angular/forms';
import { Router } from '@angular/router';
// Diálogo de Material: directivas (mat-dialog-title, -content, -actions) y
tokens
import { MAT_DIALOG_DATA, MatDialogModule, MatDialogRef } from
'@angular/material/dialog';
import { MatSnackBar } from '@angular/material/snack-bar';
import { MatFormFieldModule } from '@angular/material/form-field';
import { MatInputModule } from '@angular/material/input';
import { MatButtonModule } from '@angular/material/button';
import { Producto } from '../../models/producto.model';
import { ProductoService } from '../../services/producto.service';

/**
 * Formulario de creación y edición de productos.
 * Normalmente se muestra dentro de un diálogo de Angular Material.
 */
@Component({
  selector: 'app-producto-form',
  imports: [
    ReactiveFormsModule,
    MatDialogModule,
    MatFormFieldModule,
    MatInputModule,
    MatButtonModule
  ],
  templateUrl: './producto-form.component.html',
  styleUrl: './producto-form.component.scss'
```

```
})
export class ProductoFormComponent {
  private readonly fb = inject(FormBuilder);
  private readonly productoService = inject(ProductoService);
  private readonly snackBar = inject(MatSnackBar);
  private readonly router = inject(Router);

  // { optional: true } -> null si el componente no se abrió en un diálogo
(ruta /add)
  readonly dialogRef = inject<MatDialogRef<ProductoFormComponent>>
(MatDialogRef, { optional: true });
  private readonly data = inject<Partial<Producto> | null>(MAT_DIALOG_DATA, {
optional: true });

  // Producto recibido (vacío al crear)
  private readonly producto: Partial<Producto> = this.data ?? {};

  // ¿Editando (true) o creando (false)? Edición si el producto tiene ID
  readonly isEditing = !!this.producto.id;

  // Formulario reactivo con valores iniciales y validadores
  readonly productoForm = this.fb.nonNullable.group({
    nombre: [this.producto.nombre ?? '', [Validators.required,
Validators.minLength(3)]],
    descripcion: [this.producto.descripcion ?? '', [Validators.required]],
    precio: [this.producto.precio ?? 0, [Validators.required,
Validators.min(0)]],
    stock: [this.producto.stock ?? 0, [Validators.required,
Validators.min(0)]]
  });

  /** Crea o actualiza según el modo */
  onSubmit(): void {
    if (this.productoForm.invalid) {
      return;
    }

    // getRawValue() devuelve { nombre, descripcion, precio, stock } con
tipos
    const producto: Producto = this.productoForm.getRawValue();

    if (this.isEditing) {
      this.productoService.update(this.producto.id!, producto).subscribe({
        next: () => {
          this.snackBar.open('Producto actualizado correctamente', 'Cerrar',
{ duration: 3000 });
```

```
          this.cerrar(true);
        },
        error: e => console.error(e)
      });
    } else {
      this.productoService.create(producto).subscribe({
        next: () => {
          this.snackBar.open('Producto creado correctamente', 'Cerrar', {
duration: 3000 });
          this.cerrar(true);
        },
        error: e => console.error(e)
      });
    }
  }

  /** Cancela sin devolver resultado */
  cancelar(): void {
    this.cerrar();
  }

  /** Cierra el diálogo o, si no hay diálogo, vuelve a la lista */
  private cerrar(resultado?: boolean): void {
    if (this.dialogRef) {
      this.dialogRef.close(resultado);
    } else {
      this.router.navigate(['/productos']);
    }
  }
}
```

El PDF creaba el formulario en `ngOnInit`; aquí se crea al declarar la propiedad, porque `inject()` ya ha resuelto `data` en ese momento. Los errores y el botón deshabilitado se actualizan bien con `OnPush`: cada cambio llega por un evento del propio componente (teclear, salir del campo).

5.4 Formulario: `producto-form.component.html`

```
<!-- src/app/components/producto-form/producto-form.component.html -->

<!-- Título que cambia según editemos o creemos -->
<h2 mat-dialog-title>{{ isEditing ? 'Editar' : 'Nuevo' }} Producto</h2>

<form [formGroup]="productoForm" (ngSubmit)="onSubmit()">
  <div mat-dialog-content>
```

```
    <!-- Nombre -->
    <mat-form-field appearance="outline" class="w-100">
      <mat-label>Nombre</mat-label>
      <input matInput formControlName="nombre" required>
      @if (productoForm.controls.nombre.hasError('required')) {
        <mat-error>El nombre es obligatorio</mat-error>
      }
      @if (productoForm.controls.nombre.hasError('minlength')) {
        <mat-error>El nombre debe tener al menos 3 caracteres</mat-error>
      }
    </mat-form-field>

    <!-- Descripción -->
    <mat-form-field appearance="outline" class="w-100">
      <mat-label>Descripción</mat-label>
      <textarea matInput formControlName="descripcion" rows="3" required>
</textarea>
      @if (productoForm.controls.descripcion.hasError('required')) {
        <mat-error>La descripción es obligatoria</mat-error>
      }
    </mat-form-field>

    <!-- Precio y stock en la misma fila (Bootstrap) -->
    <div class="row">
      <div class="col-md-6">
        <mat-form-field appearance="outline" class="w-100">
          <mat-label>Precio</mat-label>
          <span matTextPrefix>$ </span>
          <input matInput type="number" formControlName="precio" min="0"
required>
          @if (productoForm.controls.precio.hasError('required')) {
            <mat-error>El precio es obligatorio</mat-error>
          }
          @if (productoForm.controls.precio.hasError('min')) {
            <mat-error>El precio debe ser mayor o igual a 0</mat-error>
          }
        </mat-form-field>
      </div>

      <div class="col-md-6">
        <mat-form-field appearance="outline" class="w-100">
          <mat-label>Stock</mat-label>
          <input matInput type="number" formControlName="stock" min="0"
required>
          @if (productoForm.controls.stock.hasError('required')) {
```

```
            <mat-error>El stock es obligatorio</mat-error>
          }
          @if (productoForm.controls.stock.hasError('min')) {
            <mat-error>El stock debe ser mayor o igual a 0</mat-error>
          }
        </mat-form-field>
      </div>
    </div>
  </div>

  <!-- Botones de acción -->
  <div mat-dialog-actions class="justify-content-end">
    <!-- type="button" para que no envíe el formulario -->
    <button matButton type="button" (click)="cancelar()">Cancelar</button>
    <button matButton="elevated" color="primary" type="submit"
[disabled]="productoForm.invalid">
      {{ isEditing ? 'Actualizar' : 'Guardar' }}
    </button>
  </div>
</form>
```

Con un formulario tipado (`fb.nonNullable.group`) se accede a cada campo con `productoForm.controls.nombre` en lugar de `productoForm.get('nombre')?` ; así lo comprueba `strictTemplates` , activado por defecto en Angular 22.

5.5 Detalle: `producto-detalle.component.ts`

```
// src/app/components/producto-detalle/producto-detalle.component.ts

import { Component, inject, OnInit, signal } from '@angular/core';
import { FormsModule } from '@angular/forms';
import { ActivatedRoute, Router, RouterLink } from '@angular/router';
import { MatSnackBar } from '@angular/material/snack-bar';
import { MatButtonModule } from '@angular/material/button';
import { Producto } from '../../models/producto.model';
import { ProductoService } from '../../services/producto.service';

/**
 * Muestra y modifica los detalles de un producto en una página propia (no en
diálogo).
 */
@Component({
  selector: 'app-producto-detalle',
  imports: [FormsModule, RouterLink, MatButtonModule],
  templateUrl: './producto-detalle.component.html',
  styleUrl: './producto-detalle.component.scss'
```

```
})
export class ProductoDetalleComponent implements OnInit {
  private readonly productoService = inject(ProductoService);
  private readonly route = inject(ActivatedRoute);
  private readonly router = inject(Router);
  private readonly snackBar = inject(MatSnackBar);

  // Producto actual (null mientras carga) y mensaje de error, en signals
  readonly currentProducto = signal<Producto | null>(null);
  readonly message = signal('');

  /** Lee el :id de la URL y carga el producto */
  ngOnInit(): void {
    this.obtenerProducto(Number(this.route.snapshot.paramMap.get('id')));
  }

  obtenerProducto(id: number): void {
    this.productoService.get(id).subscribe({
      next: data => this.currentProducto.set(data),
      error: e => {
        console.error(e);
        this.message.set('No se encontró el producto.');
      }
    });
  }

  actualizarProducto(): void {
    const producto = this.currentProducto();
    if (!producto?.id) return;

    this.productoService.update(producto.id, producto).subscribe({
      next: () => {
        this.snackBar.open('Producto actualizado correctamente', 'Cerrar', {
duration: 3000 });
      },
      error: e => console.error(e)
    });
  }

  eliminarProducto(): void {
    const producto = this.currentProducto();
    if (!producto?.id) return;

    if (confirm('¿Está seguro de eliminar este producto?')) {
      this.productoService.delete(producto.id).subscribe({
        next: () => {
```

```
          this.router.navigate(['/productos']);
          this.snackBar.open('Producto eliminado correctamente', 'Cerrar', {
duration: 3000 });
        },
        error: e => console.error(e)
      });
    }
  }
}
```

5.6 Detalle: `producto-detalle.component.html`

El PDF muestra esta plantilla hasta el campo *Stock* (pág. 15); los botones del final y el mensaje de error son mi propuesta para completarla según los métodos del componente. `@if (...; as p)` da un nombre al producto y los `[(ngModel)]` editan sus campos directamente.

```
<!-- src/app/components/producto-detalle/producto-detalle.component.html -->

<div class="container mt-4">
  <div class="row justify-content-center">
    <div class="col-md-8">
      <div class="card">
        <div class="card-header bg-primary text-white">
          <h3>Detalle del Producto</h3>
        </div>

        <div class="card-body">
          @if (currentProducto(); as p) {
            <div class="mb-3">
              <div class="form-group mb-2">
                <label for="nombre">Nombre</label>
                <input type="text" class="form-control" id="nombre"
                       [(ngModel)]="p.nombre" name="nombre" />
              </div>

              <div class="form-group mb-2">
                <label for="descripcion">Descripción</label>
                <textarea class="form-control" id="descripcion"
                          [(ngModel)]="p.descripcion" name="descripcion">
</textarea>
              </div>

              <div class="form-group mb-2">
                <label for="precio">Precio</label>
```

```
                <input type="number" class="form-control" id="precio"
                       [(ngModel)]="p.precio" name="precio" />
              </div>

              <div class="form-group mb-2">
                <label for="stock">Stock</label>
                <input type="number" class="form-control" id="stock"
                       [(ngModel)]="p.stock" name="stock" />
              </div>
            </div>

            <div class="d-flex gap-2">
              <button matButton="elevated" color="primary"
(click)="actualizarProducto()">Actualizar</button>
              <button matButton="elevated" color="warn"
(click)="eliminarProducto()">Eliminar</button>
              <a matButton routerLink="/productos">Volver</a>
            </div>
          } @else {
            <p>{{ message() || 'Cargando producto…' }}</p>
            <a matButton routerLink="/productos">Volver a la lista</a>
          }
        </div>
      </div>
    </div>
  </div>
</div>
```

Opcional: en la lista, cambia la celda del nombre por `<a [routerLink]="['/productos', producto.id]">{{ producto.nombre }}</a>` (e importa `RouterLink` en `ProductosListComponent`) para llegar al detalle con un clic; en el PDF solo se accedía escribiendo la URL.

Paso 6: Configuración, rutas y componente raíz

Lo que el PDF ponía en `app.module.ts` desaparece casi entero: los módulos de Material van en el `imports` de cada componente (Paso 5), HttpClient ya viene incluido desde Angular 21 y no hay `BrowserAnimationsModule`.

6.1 `src/app/app.config.ts`

No hay que tocarlo. El CLI 22 lo genera más o menos así:

```
// src/app/app.config.ts
import { ApplicationConfig, provideBrowserGlobalErrorListeners } from
'@angular/core';
import { provideRouter } from '@angular/router';

import { routes } from './app.routes';

export const appConfig: ApplicationConfig = {
  providers: [
    provideBrowserGlobalErrorListeners(),
    provideRouter(routes)
  ]
};
```

No aparece `provideZoneChangeDetection()` porque la app es *zoneless*, ni `provideHttpClient()` porque ya no es necesario. Si aun así ves `NullInjectorError: No provider for HttpClient`, añade `provideHttpClient()` (de `@angular/common/http`) a `providers`.

6.2 Rutas: `src/app/app.routes.ts`

Mismas rutas que el `app-routing.module.ts` del PDF:

```
// src/app/app.routes.ts
import { Routes } from '@angular/router';
import { ProductosListComponent } from './components/productos-
list/productos-list.component';
import { ProductoDetalleComponent } from './components/producto-
detalle/producto-detalle.component';
import { ProductoFormComponent } from './components/producto-form/producto-
form.component';

export const routes: Routes = [
  { path: '', redirectTo: 'productos', pathMatch: 'full' },
  { path: 'productos', component: ProductosListComponent },
  { path: 'productos/:id', component: ProductoDetalleComponent },
  { path: 'add', component: ProductoFormComponent },
  { path: '**', redirectTo: 'productos' } // añadida: cualquier ruta
desconocida vuelve a la lista
];
```

6.3 Componente raíz: `src/app/app.ts`

Sustituye el contenido generado por:

```
// src/app/app.ts
import { Component } from '@angular/core';
import { RouterLink, RouterLinkActive, RouterOutlet } from '@angular/router';

@Component({
  selector: 'app-root',
  imports: [RouterOutlet, RouterLink, RouterLinkActive],
  templateUrl: './app.html',
  styleUrl: './app.scss'
})
export class App {}
```

6.4 Plantilla raíz: `src/app/app.html`

Borra todo el contenido de ejemplo que genera el CLI y pon la barra de navegación del PDF:

```
<!-- src/app/app.html -->
<nav class="navbar navbar-expand-lg navbar-dark bg-primary">
  <div class="container">
    <a class="navbar-brand" routerLink="/">CRUD Angular-MySQL</a>
    <button class="navbar-toggler" type="button" data-bs-toggle="collapse"
data-bs-target="#navbarNav"
            aria-controls="navbarNav" aria-expanded="false" aria-
label="Mostrar menú">
      <span class="navbar-toggler-icon"></span>
    </button>
    <div class="collapse navbar-collapse" id="navbarNav">
      <ul class="navbar-nav">
        <li class="nav-item">
          <a class="nav-link" routerLink="/productos"
routerLinkActive="active">Productos</a>
        </li>
      </ul>
    </div>
  </div>
</nav>

<div class="container-fluid">
  <router-outlet />
</div>
```

El botón hamburguesa (`data-bs-toggle`) necesita el JavaScript de Bootstrap, que el PDF no cargaba. Si quieres que funcione en pantallas estrechas, añade en `angular.json` → `build → options` :

```
"scripts": ["node_modules/bootstrap/dist/js/bootstrap.bundle.min.js"]
```

6.5 `src/main.ts`

No hay que tocarlo; el CLI ya lo genera así:

```
// src/main.ts
import { bootstrapApplication } from '@angular/platform-browser';
import { appConfig } from './app/app.config';
import { App } from './app/app';

bootstrapApplication(App, appConfig)
  .catch((err) => console.error(err));
```

6.6 Opcional: precios en euros con formato español

El PDF muestra los precios en dólares (`$1,299.99`). Para ver `1.299,99 €` :

```
// src/main.ts (añade antes de bootstrapApplication)
import { registerLocaleData } from '@angular/common';
import localeEs from '@angular/common/locales/es';
registerLocaleData(localeEs);
```

```
// src/app/app.config.ts (añade a providers)
import { DEFAULT_CURRENCY_CODE, LOCALE_ID } from '@angular/core';
// ...
{ provide: LOCALE_ID, useValue: 'es' },
{ provide: DEFAULT_CURRENCY_CODE, useValue: 'EUR' },
```

Y en el formulario cambia `<span matTextPrefix>$ </span>` por `<span matTextSuffix> €</span>` .

6.7 Arrancar el frontend

Con MAMP arrancado (Apache sirve el backend PHP; no hace falta lanzar ningún servidor de Node para él), desde la raíz del proyecto:

```
cd /Applications/MAMP/htdocs/angular22/crudelectronica
ng serve
```

Abre `http://localhost:4200/` .

Paso 7: Probar el CRUD

Repite las mismas pruebas de las págs. 36–39 del PDF; si las seis salen bien, la app está completa.

- [ ] **Listar.** `http://localhost:4200/` redirige a `/productos` y muestra los 5 productos con los iconos de lápiz y papelera (no texto "ed"/"de").
- [ ] **Crear.** *Nuevo Producto* → `Samsung Galaxy S10 plus`, `Teléfono móvil para personas inquietas`, precio 600, stock 10 → *Guardar*. Aparece el aviso "Producto creado correctamente" y la fila con ID 6.
- [ ] **Paginar.** Con 5 elementos por página, el producto 6 pasa a la página 2 (`6 – 6 of 6`).
- [ ] **Editar.** Lápiz del producto 6 → el diálogo se titula *Editar Producto* y el botón *Actualizar*; stock 20 → la tabla muestra 20.
- [ ] **Eliminar.** Papelera del producto 6 → confirmación "¿Está seguro de eliminar este producto?" → *Aceptar*. Quedan 5 productos.
- [ ] **Filtrar.** Escribe `Sony` y pulsa la lupa o Enter → solo aparece *Auriculares Sony WH-1000XM5*. Borra el texto y busca de nuevo → vuelven todos.

Comprobaciones extra que el PDF no hacía:

- [ ] **Detalle.** `http://localhost:4200/productos/1` muestra la tarjeta del portátil; *Actualizar* guarda y *Eliminar* vuelve a la lista.
- [ ] **Ruta /add.** `http://localhost:4200/add` muestra el formulario fuera del diálogo y, al guardar o cancelar, vuelve a `/productos`.
- [ ] **Validaciones.** Nombre de 2 letras o precio negativo → el botón *Guardar* queda deshabilitado y aparece el mensaje de error.
- [ ] **Tildes.** Crea `Ratón inalámbrico` y comprueba en phpMyAdmin que se guarda sin caracteres raros.
- [ ] **Repintado sin zone.js.** Tras crear, editar, borrar o buscar, la tabla y el contador del paginador cambian sin recargar la página. Si no lo hacen, algún dato que llega del servidor no se está guardando en un signal (Paso 5).

Los textos del paginador ("Items per page", "of") salen en inglés, igual que en el PDF. Para traducirlos hay que proporcionar un `MatPaginatorIntl` propio, como hace el `PaginatorI18nService` del repositorio.

Paso 8: Desplegar en Arsys

La app se sube como ficheros estáticos y el backend como PHP; Arsys no necesita Node. Hay cuatro sitios que deben apuntar a las mismas carpetas:

Dónde	Qué	Ejemplo
`angular.json` → `configurations.production.baseHref`	Carpeta de la app	`/angular/crudmysqlelectronica/`
`.htaccess` de la app → `RewriteBase`	La misma carpeta	`/angular/crudmysqlelectronica/`
`src/environments/environment.ts` → `apiUrl`	`index.php` del backend	`/angular/crudelectronicasimplificado-backend/index.php`
`backend-php/index.php` → `$__allowed`	Orígenes CORS	ya incluye `txemaserrano.com` (http y https)

Las rutas del ejemplo son las que tienes publicadas ahora; el repo usa todavía `crudmysqlelebiduna`. Cámbialas si despliegas en otra carpeta.

8.1 Compilar

En `angular.json`, dentro de `architect → build → configurations → production`, añade:

```
"baseHref": "/angular/crudmysqlelectronica/"
```

```
ng build
```

Con el compilador actual la salida queda en `dist/crud-angular-mysql/browser/` (en el repo de Angular 15 era `dist/crud-angular-mysql/`). Lo que se sube es el **contenido** de `browser/`.

8.2 `.htaccess` de la app

Copia `crudelectronica-15/src/htaccess` a `src/htaccess` del proyecto nuevo y ajusta `RewriteBase`. Contenido (del repo):

```
# Va en la MISMA carpeta que index.html y main.*.js
# Súbelo por FTP y renómbralo en el servidor a exactamente: .htaccess

# 1) Declara el MIME correcto de los módulos JS y el CSS.
<IfModule mod_mime.c>
  AddType application/javascript .js
  AddType text/javascript        .mjs
  AddType text/css               .css
</IfModule>

# 2) Refuerzo: fuerza el tipo aunque algo intente servirlo como text/html
```

```
#    (resuelve el error "Expected a JavaScript module script").
<FilesMatch "\.js$">
  ForceType application/javascript
</FilesMatch>
<FilesMatch "\.mjs$">
  ForceType text/javascript
</FilesMatch>
<FilesMatch "\.css$">
  ForceType text/css
</FilesMatch>

# 3) Enrutado: sirve los ficheros reales; el resto -> index.html (router
Angular).
<IfModule mod_rewrite.c>
  RewriteEngine On
  RewriteBase /angular/crudmysqlelectronica/

  RewriteCond %{REQUEST_FILENAME} -f [OR]
  RewriteCond %{REQUEST_FILENAME} -d
  RewriteRule ^ - [L]

  RewriteRule ^ index.html [L]
</IfModule>
```

Sin la regla del punto 3, recargar `/angular/crudmysqlelectronica/productos/3` da un 404.

8.3 Subir por FTP (FileZilla)

1. El contenido de `dist/crud-angular-mysql/browser/` → `/angular/crudmysqlelectronica/`.
2. `src/htaccess` → esa misma carpeta, y renómbralo a `.htaccess` en el servidor.
3. El contenido de `backend-php/` → `/angular/crudelectronicasimplificado-backend/`, con su `.htaccess`. **No subas** `sql/` si no hace falta, ni ficheros `._*`.
4. Crea en el servidor `config.local.php` (plantilla del apartado 2.2) con las credenciales de MySQL de Arsys. Este fichero solo existe allí.
5. Si la base de producción aún no tiene la tabla, importa `sql/crud.sql` desde el phpMyAdmin del hosting. Si la base ya existe con otro nombre, quita antes del script las líneas `CREATE DATABASE` y `USE`.

8.4 Comprobar

- `https://txemaserrano.com/angular/crudelectronicasimplificado-backend/index.php/productos` devuelve el JSON de productos.

- `https://txemaserrano.com/angular/crudmysqlelectronica/` carga la app y lista los productos.
- Recargar el navegador estando en `/productos/1` sigue mostrando el detalle (prueba el `.htaccess`).

Errores frecuentes y soluciones

La tabla recoge los errores del PDF que siguen aplicando, los del backend PHP (según el README del repo) y los más probables al pasar a Angular 22.

Síntoma	Causa probable	Solución
La tabla no cambia tras guardar o borrar, pero al recargar sí aparece	*Zoneless* + `OnPush`: el dato se guardó en una propiedad normal	Guárdalo con `signal.set()` (Paso 5)
`500 Error de base de datos` desde la API	Host, puerto o contraseña incorrectos en `config.php` / `config.local.php`	En MAMP: `127.0.0.1`, puerto `8889`, `root` / `root`
`500` con `No such file or directory` en el mensaje de PDO	`host` = `localhost` hace que PDO use el socket, no el puerto	Cambia a `127.0.0.1`
`404 Ruta no encontrada` desde la API	Falta `/productos` en la URL o no se aplica el `.htaccess`	Usa `.../index.php/productos` o `?path=productos`; renombra `htaccess` a `.htaccess`
Error de CORS en la consola del navegador	El origen no está en `$__allowed` de `index.php`	Añádelo (p. ej. si sirves la app en otro puerto que no sea 4200)
"Expected a JavaScript module script but the server responded with a MIME type of text/html" en Arsys	Falta el `.htaccess` de la app o `baseHref` no coincide con la carpeta	Paso 8.2; comprueba que el fichero se llama `.htaccess`, con punto
404 al recargar `/productos/3` en producción	Falta la regla de reescritura a `index.html`	`RewriteBase` del `.htaccess` de la app (Paso 8.2)
Tildes o eñes mal guardadas	Importación sin `utf8mb4`	Reimporta `crud.sql`, que empieza con `SET NAMES utf8mb4`
`NG8023: Multiple components match node with tagname button` al compilar	Dos directivas de botón en el mismo elemento (en el PDF salía como NG0300)	Una sola: `mat-icon-button` (Paso 5.2)
Los iconos salen como texto ("ed", "de", "search")	Falta la fuente Material Icons	`<link>` de Material Icons en `index.html` (Paso 3.6)
`dyld: Symbol not found` al ejecutar `node`	Node 24 exige macOS 13.5; Catalina es 10.15	Usa Node 22.22.3+ si arranca; si no, Angular 20 + Node 20 u otro equipo

Síntoma	Causa probable	Solución
`The engine "node" is incompatible` o `ng` no arranca	Node anterior a 22.22.3 (por ejemplo 22.21.0)	`nvm install 22` (descarga la última 22.x)
Errores de tipos nuevos en plantillas	`strictTemplates` activado por defecto en Angular 22	Usa `productoForm.controls.campo` y tipa los datos
`NG8001: 'mat-form-field' is not a known element` (u otro `mat-*`)	El componente standalone no importa ese módulo	Añádelo al array `imports` del componente
`Can't bind to 'ngModel'` / `'formGroup'`	Falta `FormsModule` o `ReactiveFormsModule` en `imports`	Añádelo al componente que lo usa
`Cannot find module './components/.../productos-list.component'`	Componentes generados sin sufijo	Configura `schematics` (Paso 3.2) y regenera
La tabla aparece vacía y no hay error	MAMP parado o `apiUrl` incorrecta	Pestaña *Red* de las DevTools y `environment.development.ts`
El diálogo no tiene estilos	No se cargó el tema de Material	Array `styles` de `angular.json` (Paso 3.4); prueba `azure-blue.css`
`npm install` falla con `ERESOLVE`	Material/CDK en otra versión mayor que Angular	`@angular/material@22` y `@angular/cdk@22`
Error de inyección al abrir `/add`	Algún módulo del diálogo exige `MatDialogRef`	Ya es opcional en el Paso 5.3; si persiste, elimina la ruta `/add`, como sugiere el README del repo

Equivalencias PDF / repo → esta guía (Angular 22)

La lógica de la app no cambia; cambian el backend, dónde se declara cada cosa y cómo se guarda el estado.

En el PDF o el repo	En esta guía	Motivo
Backend Node/Express en `backEnd/` (PDF)	Backend PHP/PDO en `backend-php/` (repo)	Es el que usa el repo y corre en Arsys sin Node
`.env` + `db.config.js`	`config.php` (sin secretos) + `config.local.php` (fuera de Git)	Credenciales fuera del repositorio
`server.js` + rutas + controlador	`index.php` (router REST + CORS)	Mismo contrato de API

En el PDF o el repo	En esta guía	Motivo
`decimalNumbers` / precio como texto	`hydrate()` convierte a número en PHP	Mismo resultado
Proyecto en `frontEnd/`	Angular en la raíz y `backend-php/` dentro	Estructura del repo
zone.js + detección `Default`	*Zoneless* + `OnPush` por defecto	Defecto de las apps nuevas desde v21/v22
`MatTableDataSource` asignado en `subscribe`	`signal<Producto[]>` + `computed` para la página	Sin zone.js solo repinta lo que lee signals
`@ViewChild(MatPaginator)` + `ngAfterViewInit`	`[length]`, `[pageIndex]`, `[pageSize]` y `(page)`	Paginación controlada por signals
`app.module.ts` / `app-routing.module.ts`	`app.config.ts` + `app.routes.ts` + `imports` por componente	Componentes *standalone*
`HttpClientModule` / `BrowserAnimationsModule`	Nada	HttpClient incluido desde v21; Material no necesita animaciones
`@Injectable({ providedIn: 'root' })` + constructor	`@Service()` + `inject()`	Nuevo decorador de v22
`this.fb.group(...)` y `get('nombre')?`	`fb.nonNullable.group(...)` y `controls.nombre`	Formularios tipados + `strictTemplates`
`app.component.ts` / `AppComponent`	`app.ts` / `App`	Guía de estilo del CLI
`*ngIf`	`@if { } @else { }`	Directivas estructurales obsoletas desde v20
`mat-raised-button` / `mat-button`	`matButton="elevated"` / `matButton`	Nuevas directivas de botón desde Material 20
`@import` de Bootstrap y tema en `styles.scss`	Array `styles` de `angular.json`	`@import` de Sass obsoleto

En el PDF o el repo	En esta guía	Motivo
`environment.ts` (dev) + `environment.prod.ts`	`environment.development.ts` + `environment.ts` (prod)	Convención de `ng generate environments`
`dist/crud-angular-mysql/`	`dist/crud-angular-mysql/browser/`	Salida del compilador actual
Karma + Jasmine	Vitest (omitido con `--skip-tests`)	Defecto del CLI

Notas de verificación

Comprobado: el código del backend PHP, el script SQL, los `environments` y el `.htaccess` proceden del repositorio, que he clonado (último commit `cae548c`). Versiones y fechas de Angular 22 en angular.dev; `OnPush` por defecto, `@Service()`, Fetch por defecto, HttpClient incluido desde v21, `strictTemplates` y NG8023 en el artículo de Ninja Squad sobre 22.0; el mínimo de macOS 13.5 de Node 24 en la guía de migración de Node.js (y en el error de tu equipo).

No verificado (pruébalo antes de llevarlo a clase):

- No he podido ejecutar ni compilar el código Angular ni el PHP.
- Que la última versión de Node 22 arranque en macOS Catalina.
- Que `@angular/material@22` siga publicando el tema `indigo-pink.css` (confirmado hasta la 21.2.5).
- Las preguntas exactas de `ng new` en el CLI 22 y la forma exacta del `app.config.ts` que genera.
- Que MAMP aplique por defecto el `.htaccess` de `backend-php` (si no, usa `index.php/productos`).

Seguridad. El `backend-php/config.php` publicado en el repositorio contiene las credenciales reales de la base de datos de producción. El apartado 2.2 explica cómo dejarlo sin secretos; además, hay que cambiar esa contraseña en Arsys, porque borrarla del fichero no la quita del historial de Git.

Fuentes

- Repositorio jsersan/crudelectronica (código clonado y README)
- Angular — Version compatibility
- Angular — Versioning and releases

- Ninja Squad — What's new in Angular 22.0?
- Node.js — Migración de v22 a v24 (macOS 13.5 mínimo)
- Angular CLI — incidencia #30566 sobre los sufijos de fichero
- Angular Material — PR #30638 sobre la entrada matButton
- Angular — provideAnimationsAsync (obsoleto desde v20.2)
- PDF *CRUD mySQL Angular* (adjunto)